



The Library Magic Assistant

A natural-language concierge for the catalog you already have

Patrons increasingly expect to ask for what they need in plain language and get an instant, accurate answer. The library already owns the hard assets — a rich catalog in **Polaris**, four branches, an events program, and licensed research databases. The Library Magic Assistant adds a friendly, natural-language **front door** to those assets: available 24/7, in any language, and welcoming to first-time researchers, caregivers, and job seekers. It extends the reference desk — it does not replace it.

What patrons could do

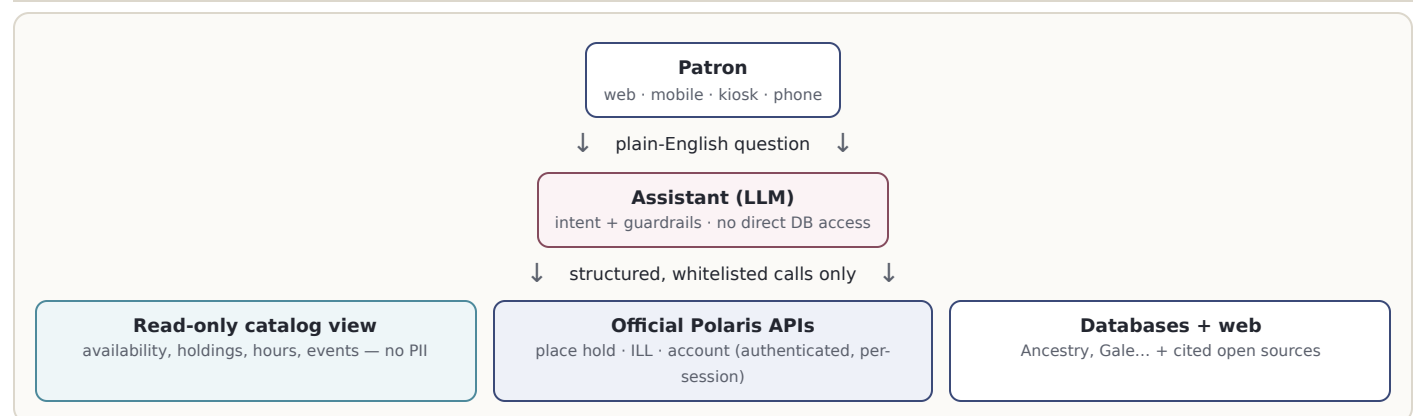
- **Find & reserve in plain English**, with live availability across all four branches.
- **Place holds** and choose a pickup branch; join the hold queue when copies are out.
- **Request interlibrary loans** when an item isn't owned.
- **Age-appropriate suggestions** with caregiver content notes.
- **Research help** — routed to the right licensed databases, augmented with *cited* sources.
- **Discover events & story times** by age, branch, or day.

All six are clickable today in the working prototype.

Trust & governance — non-negotiables

- **Read-only replica** — the assistant physically cannot alter the live ILS.
- **No patron PII** in the AI layer; account actions use official, authenticated APIs.
- **Whitelisted functions only** — the model never writes raw database queries, which defends against prompt-injection.
- **Grounded & cited** answers — no fabricated facts.
- **Full audit log**; ambiguous or sensitive requests escalate to staff.
- The existing catalog & website stay fully usable — the assistant **augments**.

How it works



The assistant interprets the question, then calls a small set of approved functions. **Reads** hit a read-only copy of catalog and availability data. **Actions** (holds, ILL, account) go through Polaris's official authenticated web services, scoped to the patron's session. **Research** routes to licensed databases and adds cited open sources — every answer carries its citations.



Phased roadmap

Phase 1 — Read-only concierge ≈ 6-10 weeks
Hours, locations, events, "is it on the shelf?", recommendations, research routing. **No writes at all.**
Proves demand and accuracy at near-zero risk.

Phase 2 — Authenticated actions next quarter
Holds, renewals, and ILL through Polaris's official APIs; library-card sign-in; staff dashboard.

Phase 3 — Reach & depth following
Multilingual, voice/phone, kiosk mode, deeper research augmentation, and analytics to inform collection decisions.

Next steps — 30 / 60 / 90 days

- 1 **Align on scope & success metrics** with leadership — define what "good" looks like at 90 days.
- 2 **Technical discovery with Polaris / Innovative** — confirm read exports, write APIs (PAPI), and database licensing terms.
- 3 **Stand up the Phase-1 pilot** on real, read-only data behind a staff login; test with reference staff.
- 4 **Soft-launch to patrons** on one high-traffic page; measure usage, accuracy, and staff time saved.
- 5 **Decide Phase 2 + hosting model** from pilot results.

Where it runs — local, cloud, or hybrid

Almost the entire system can run **on-premises**. The only piece that benefits from outside help is the heavy AI model — and even that can be local.

Component	Local?
Web app / patron UI	✓ Fully local (static site)
Read-only catalog replica	✓ Local DB fed by Polaris exports
Orchestration + guardrails	✓ Local service
Semantic search / embeddings	✓ Local (open models)
The language model	🔗 Local on a GPU box, <i>or</i> cloud API

Recommended: hybrid. Keep all data + retrieval + a local model in-building for routine questions; optionally call a cloud model for the hardest turns, sending only non-PII catalog snippets under a zero-retention agreement. Patron data stays home; answer quality stays high.

Rough cost envelope

Option	Up-front	Ongoing
Phase-1 pilot, cloud model	Integration effort	Low hundreds \$/mo at pilot volume
On-prem appliance (8-14B model + full stack)	~\$5-25k GPU server	≈ \$0 per query + staff time
Higher quality / concurrency on-prem	\$15-40k GPU	Power + maintenance

The library's existing internal stack already proves the local pattern: Postgres + vector search + a local model server. The same building blocks underpin Phase 1.

The ask: endorse a low-risk **Phase-1 pilot** on read-only data, choose a **name** for the assistant, and authorize a short technical-discovery call with the ILS vendor.